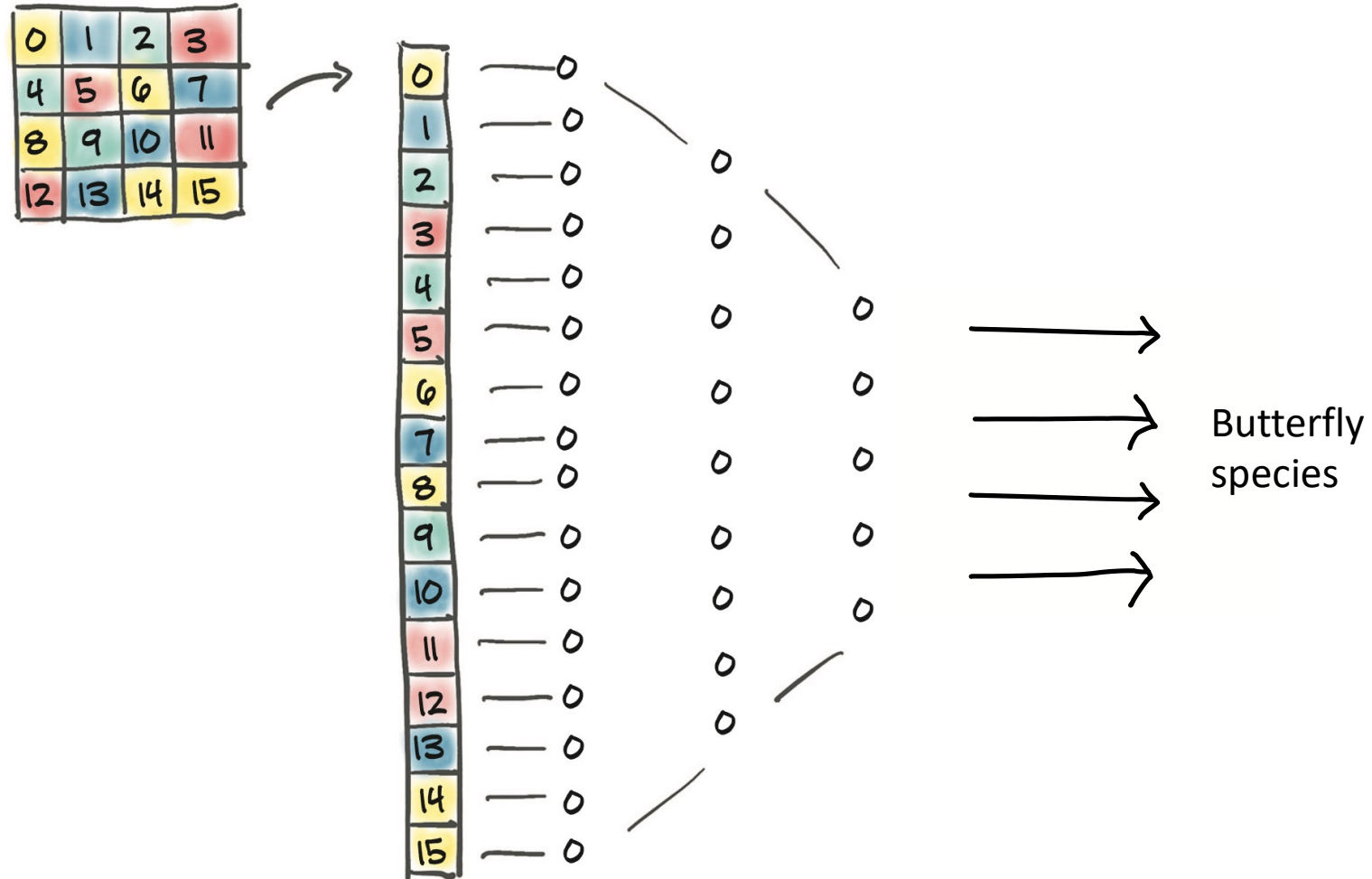


Lecture 9: Using convolutions to generalize

Learning from Images: Recap

- If we just take the image pixels and straighten them into a long 1D vector, we could consider those numbers as input features

- $3 \times 224 \times 224$: that is, 150,528 input features per sample.

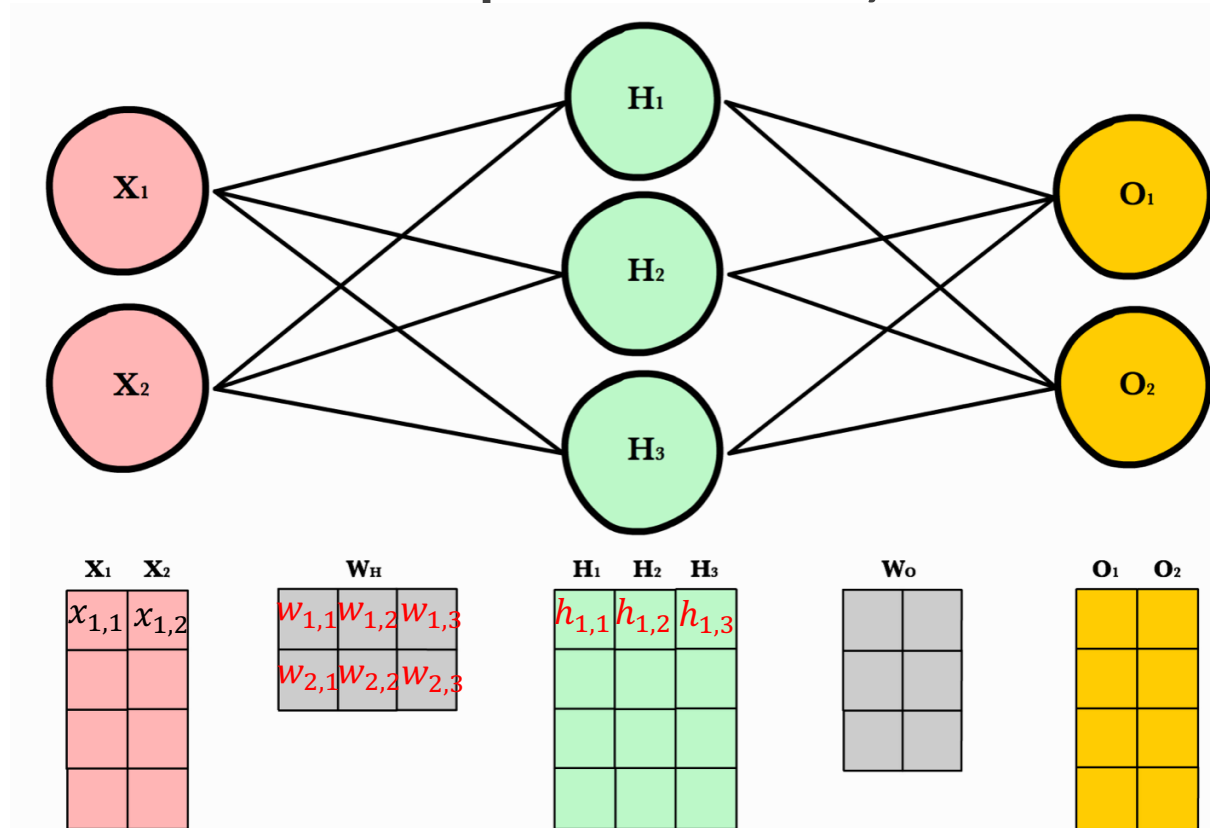


Learning from Images: Recap

- Our neural network won't scale very well with the number of pixels.
 - What if we had a $1,024 \times 1,024$ RGB image? That's 3.1 million input values. Even abruptly going to 1,024 hidden features, we would have over 3 *billion* parameters. Using 32-bit floats, we're already at 12 GB of RAM, and we haven't even hit the second layer, much less computed and stored the gradients.

Learning from Images: Recap

- A network with 2 input nodes, 3 hidden nodes, and 2 output nodes.



$$\begin{aligned} f(x_{1,1} * w_{1,1} + x_{1,2} * w_{2,1} + b_1) &\rightarrow h_{1,1} \\ f(x_{1,1} * w_{1,2} + x_{1,2} * w_{2,2} + b_2) &\rightarrow h_{1,2} \\ f(x_{1,1} * w_{1,3} + x_{1,2} * w_{2,3} + b_3) &\rightarrow h_{1,3} \end{aligned}$$

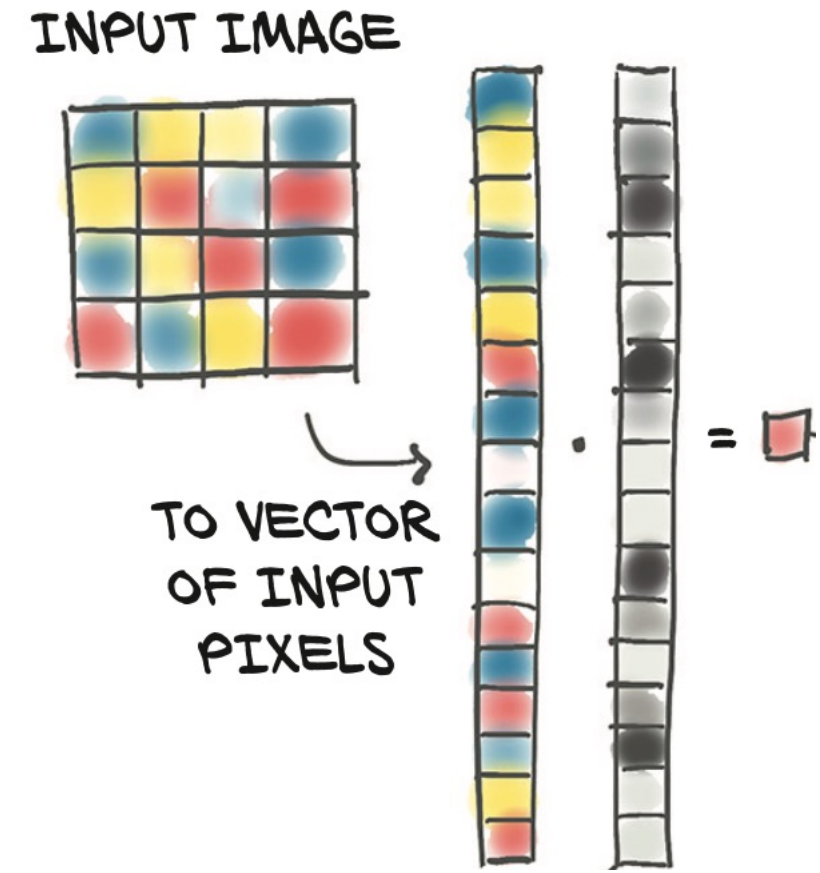
$$\mathbf{X} (n \times 2) \quad \mathbf{W} (2 \times 3) \quad \mathbf{H} (n \times 3)$$

3-dimensional hidden space

From <https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html>

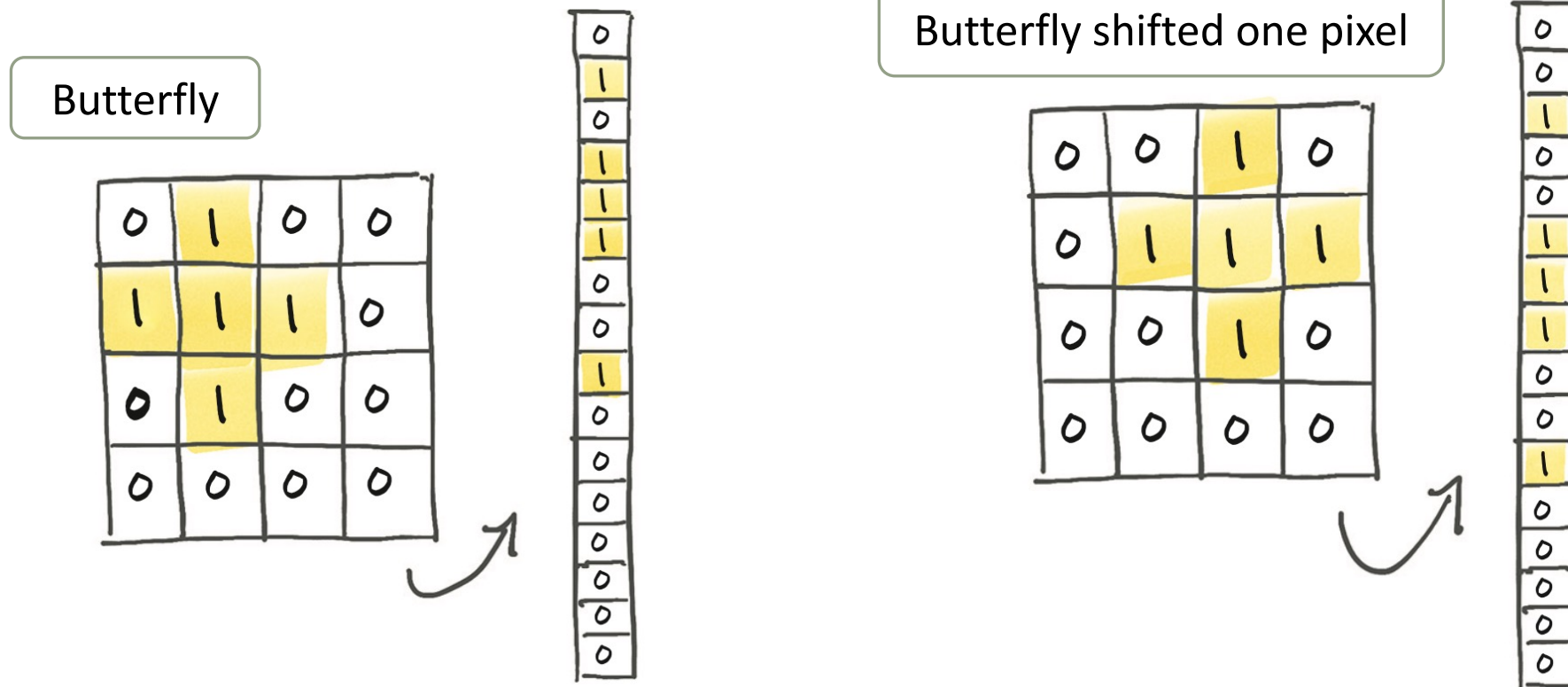
Learning from Images: Recap

- On one hand, we are allowing for the combination of any pixel with every other pixel in the image being potentially relevant for our task. On the other hand, we aren't utilizing the relative position of neighboring or faraway pixels, since we are treating the image as one big vector of numbers.



Learning from Images: Recap

- If we shift the same butterfly by one pixel or more, and the relationships between pixels will have to be relearned from scratch



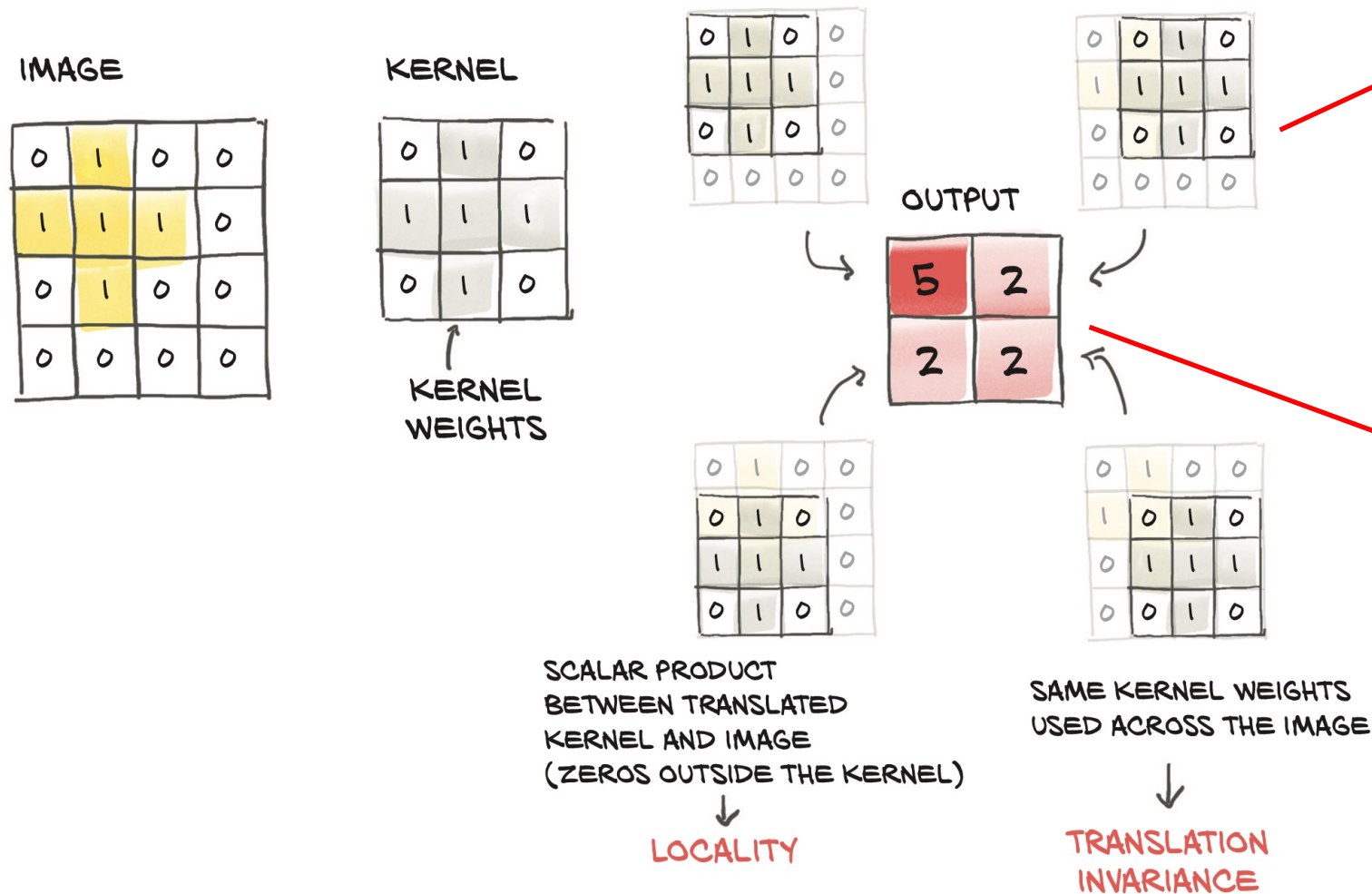
Overview

- Convolution
- Detecting Features with Convolutions
- Looking Further with Depth and Pooling
- Training on the GPU

Convolution

- The convolutional neural network (CNN) is a specialized type of neural network model designed for working with image data.
 - Convolutional layer performs a linear operation called a convolution: the multiplication between input data and an array of weights, called a filter or a kernel.

Convolution



- An element-wise multiplication between the image and filter/kernel, and then is summed, resulting in a single number/value.
- The filter is smaller than the input image, which allows the same filter/kernel to be applied to each filter-size patch, left to right, top to bottom.

<https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/>

Convolution

- Convolution is defined for a 2D image as the scalar product of a weight matrix, the *kernel*, with every neighborhood in the input.

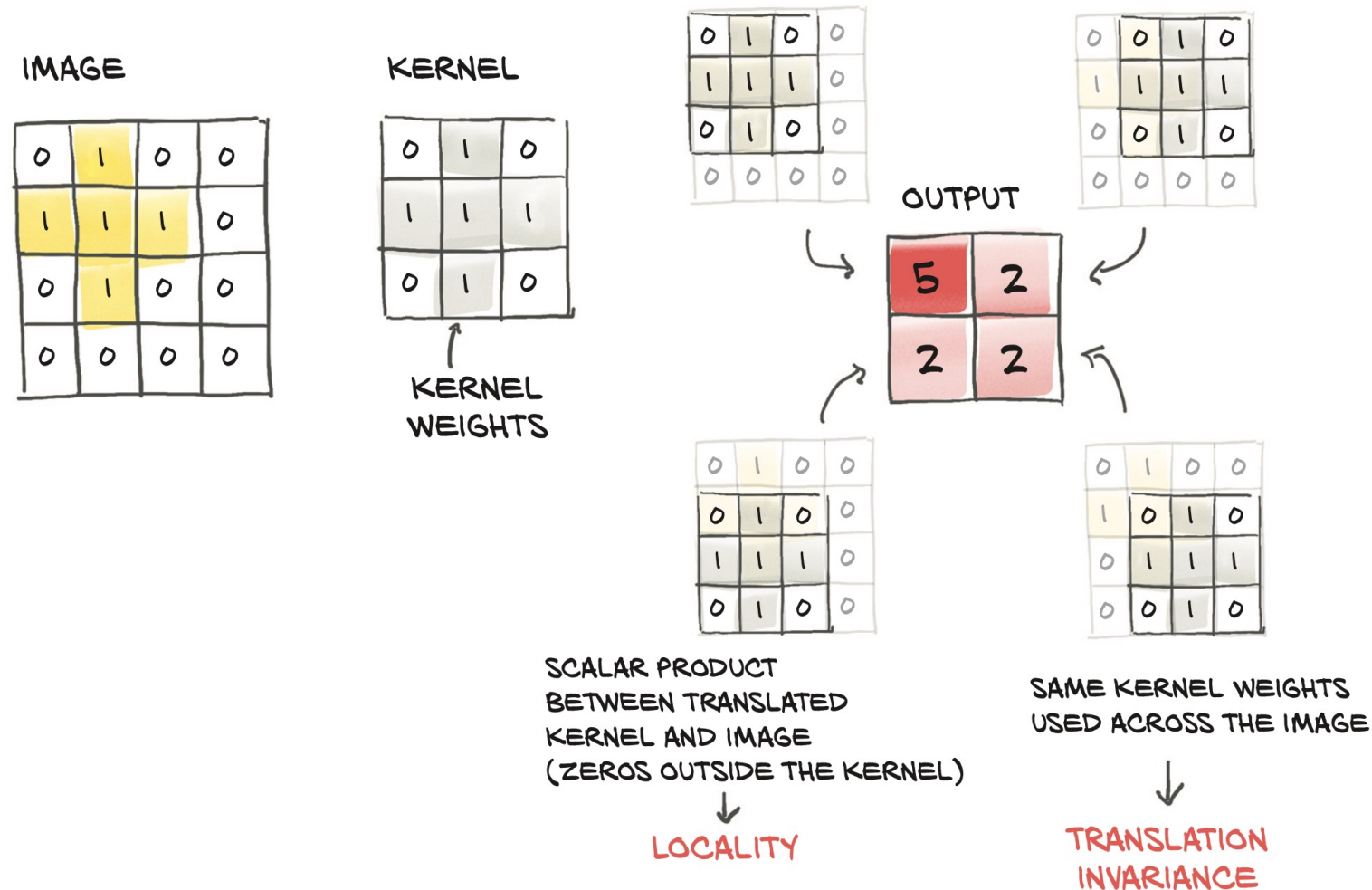
- Consider a 3×3 kernel and a 1-channel, $M \times N$ image
- We “translate” the kernel on the $i11$ location of the input image, and we multiply each weight by the value of the input image at the corresponding location.

```
weight = torch.tensor([[w00, w01, w02],  
                        [w10, w11, w12],  
                        [w20, w21, w22]])
```

```
image = torch.tensor([[i00, i01, i02, i03, ..., i0N],  
                      [i10, i11, i12, i13, ..., i1N],  
                      [i20, i21, i22, i23, ..., i2N],  
                      [i30, i31, i32, i33, ..., i3N],  
                      ...,  
                      [iM0, iM1, iM2, iM3, ..., iMN]])
```

$$\begin{aligned} o_{11} = & i_{11} * w_{00} + i_{12} * w_{01} + i_{22} * w_{02} + \\ & i_{21} * w_{10} + i_{22} * w_{11} + i_{23} * w_{12} + \\ & i_{31} * w_{20} + i_{32} * w_{21} + i_{33} * w_{22} \end{aligned}$$

Convolution



- The output image is created by translating the kernel on all input locations and performing the weighted sum.
- For a multichannel image, like our RGB image, the weight matrix would be a $3 \times 3 \times 3$ matrix: one set of weights for every channel, contributing together to the output values.

Convolution

- A convolution is equivalent to having multiple linear operations whose weights are zero almost everywhere except around individual pixels.
 - Models with a lot fewer parameters: the number of parameters depends not on the number of pixels in the image, but rather on the size of the convolution kernel and on how many convolution filters.
 - Local operations on neighborhoods
 - Translation invariance

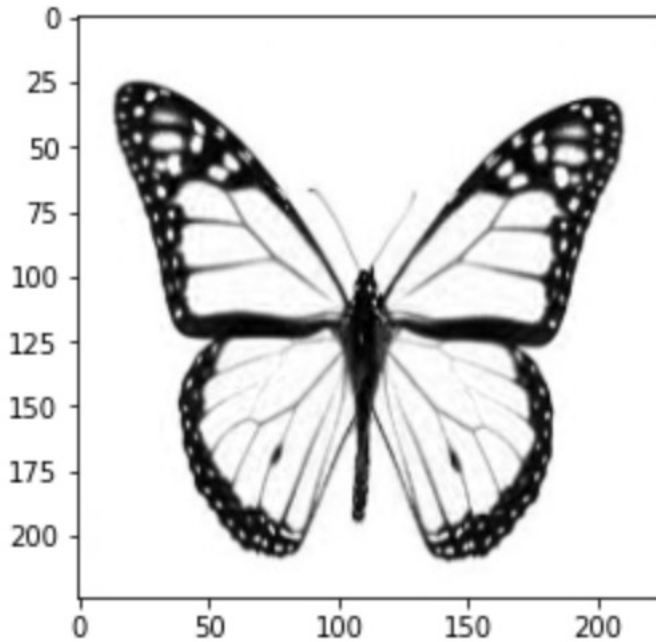
Convolution

- Translation invariance

- If the filter is designed to detect a specific type of feature in the input, then the application of that filter systematically across the entire input image allows the filter an opportunity to discover that feature anywhere in the image.
 - Number of channels: a filter has the same number of channels as the input image. For example, kernel size = 5 gives you a filter with size 3 x 5 x 5.
 - Number of filters: multiple filters to learn multiple features from the image.

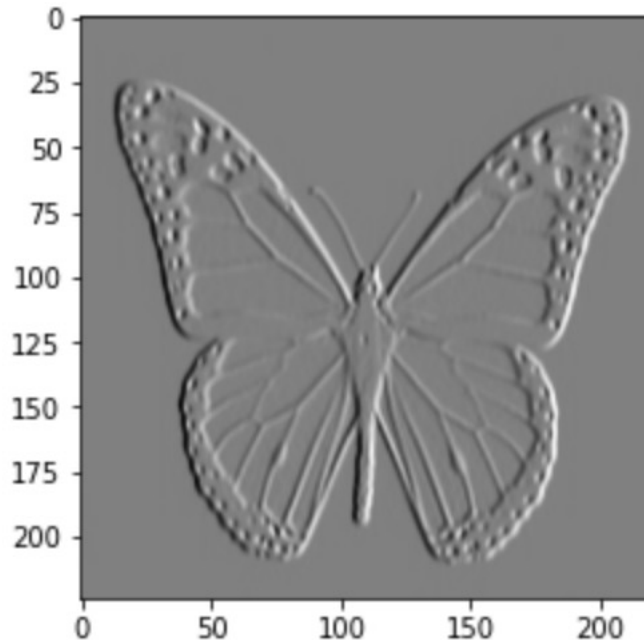
Detecting Features with Convolutions

■ Translation invariance



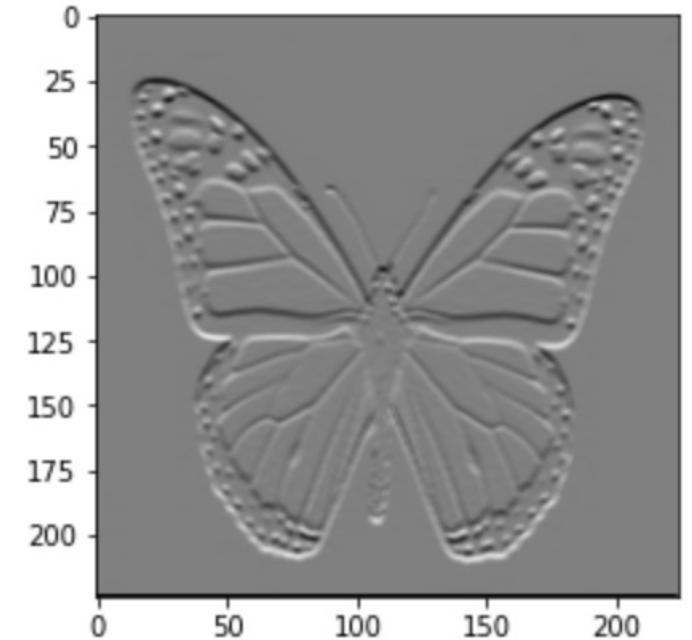
The convolution kernel enhances the vertical edges

-1	0	1
-1	0	1
-1	0	1



The convolution kernel enhances the horizontal edges

-1	-1	-1
0	0	0
1	1	1



Detecting Features with Convolutions

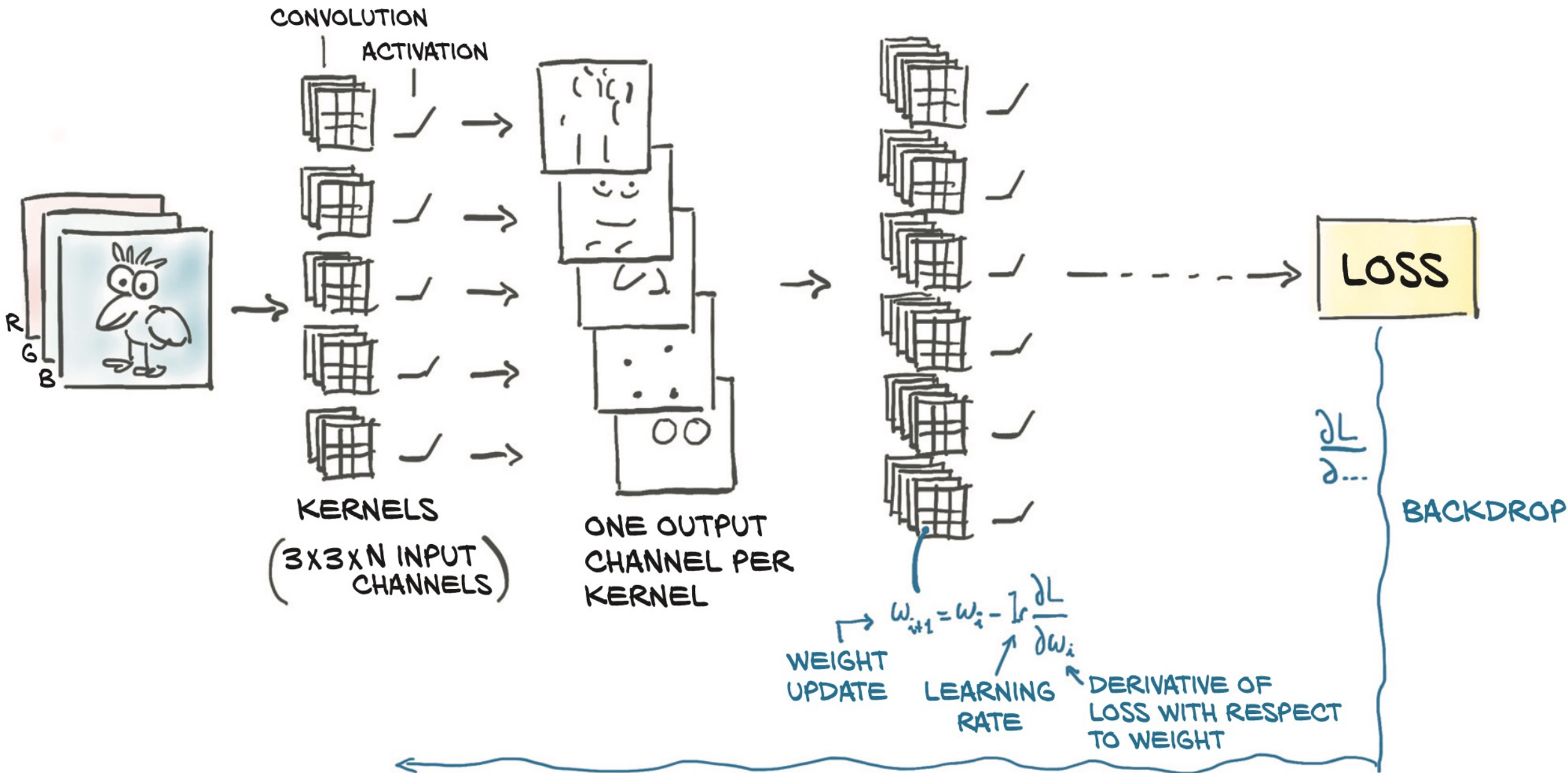
- Detecting features with convolutions
 - An *edge-detection* kernel: the kernel highlights the vertical edge between two horizontally adjacent regions.

```
conv.weight[:] = torch.tensor([[-1.0, 0.0, 1.0],  
                                [-1.0, 0.0, 1.0],  
                                [-1.0, 0.0, 1.0]])
```

- It performs the difference of all pixels on the right minus the pixels on the left. If the kernel is applied on a vertical boundary between two adjacent regions of different intensity, it will output a high value.

Detecting Features with Convolutions

- We could build lots more elaborate filters, such as for detecting horizontal or diagonal edges.
- The job of a computer vision expert has historically been to come up with the most effective combination of filters so that certain features are highlighted in images and objects can be recognized.
- We let kernels be estimated from data. The weights in the kernel are not known in advance, but they are initialized randomly and updated through backpropagation.



Detecting Features with Convolutions

- The job of a convolutional neural network is to estimate the kernel of a set of filter banks in successive layers that will transform a multichannel image into another multichannel image, where different channels correspond to different features (such as one channel for the average, another channel for vertical edges, and so on).

Detecting Features with Convolutions

```
# In[11]:
```

```
conv = nn.Conv2d(3, 16, kernel_size=3)  
conv
```

It is very common to have kernel sizes that are the same in all directions, so PyTorch has a shortcut for this: whenever `kernel_size=3`, it means 3×3 .

Instead of the shortcut `kernel_size=3`, we could equivalently pass in the tuple that we see in the output: `kernel_size=(3, 3)`.

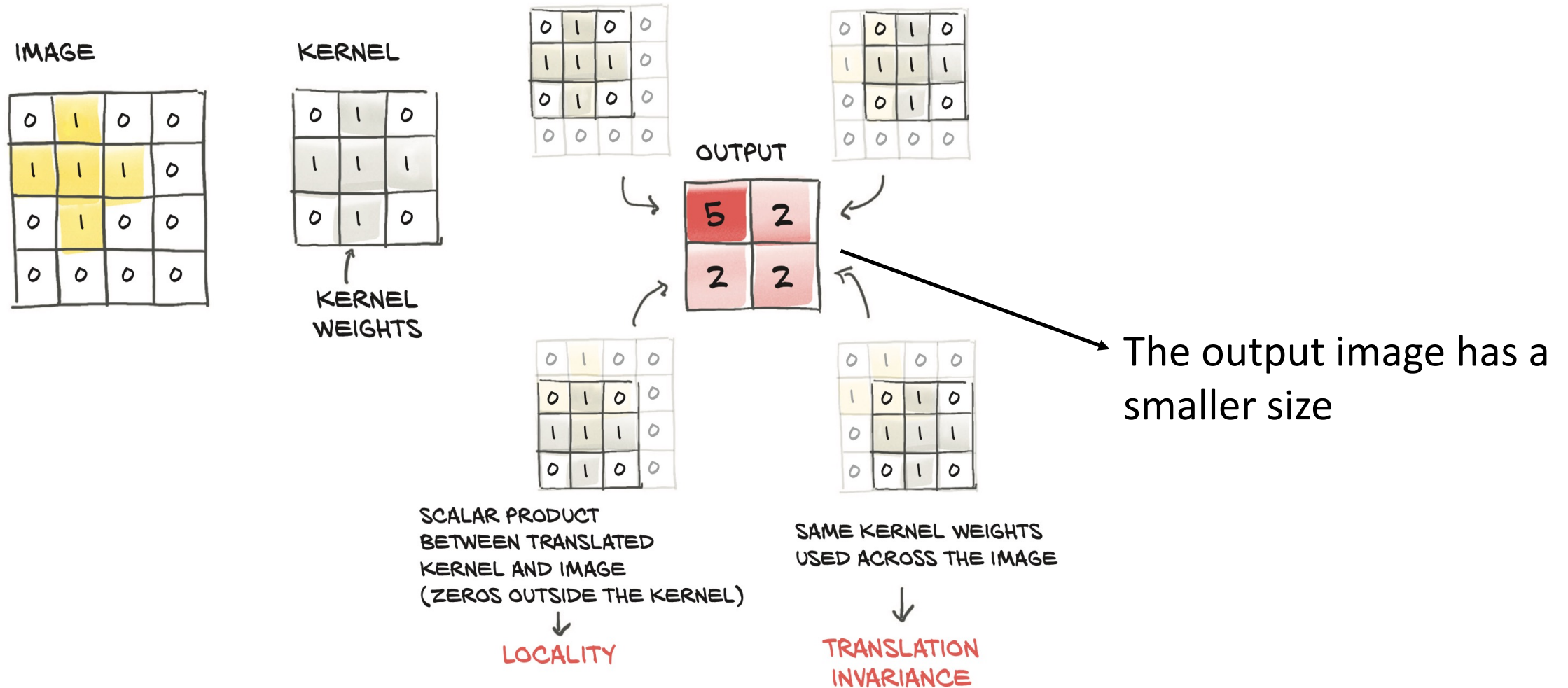
- We have 3 input features per pixel (the RGB channels) and an arbitrary number of channels in the output—say, 16. The more channels in the output image, the more the capacity of the network.

Detecting Features with Convolutions

```
# In[12]:  
conv.weight.shape, conv.bias.shape  
  
# Out[12]:  
(torch.Size([16, 3, 3, 3]), torch.Size([16]))
```

- Filters: weight tensor is $\text{out_ch} \times \text{in_ch} \times 3 \times 3$, in our case $16 \times 3 \times 3 \times 3$. (16 filters, each one is $3 \times 3 \times 3$)

Convolution-Padding the Boundary



Convolution-Padding the Boundary

- The fact that our output image is smaller than the input is a side effect of deciding what to do at the boundary of the image.
- However, PyTorch gives us the possibility of *padding* the image by creating *ghost* pixels around the border that have value zero as far as the convolution is concerned.

```
# Maintain the same image size with padding
conv = nn.Conv2d(3, 16, kernel_size=5, padding=2)
```

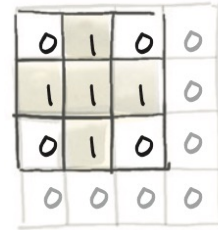
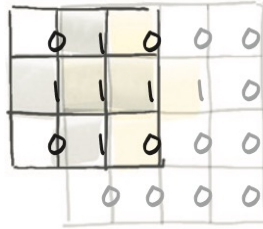
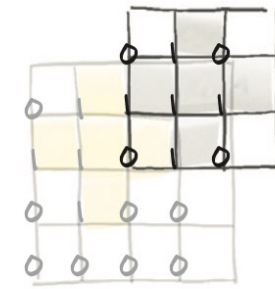
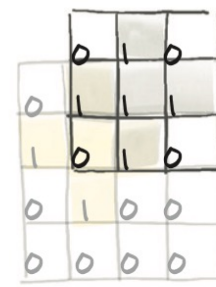
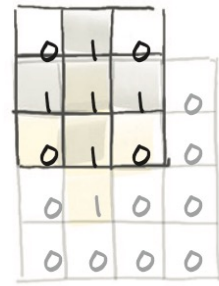
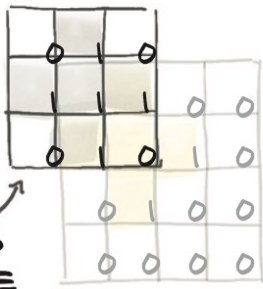
```
output = conv(img)
```

```
output.shape
```

```
torch.Size([16, 224, 224])
```

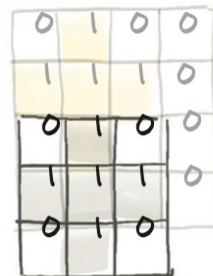
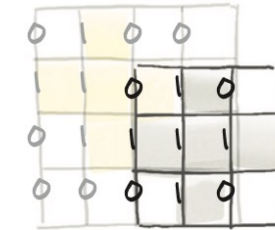
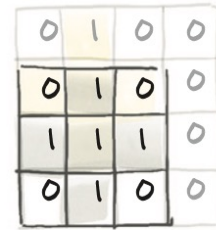
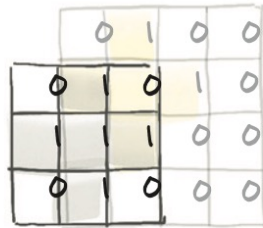
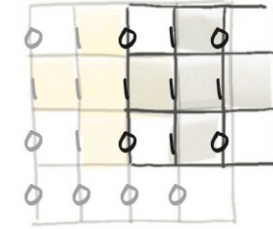
it's best to stay with odd kernel sizes; even-sized kernels are just odd (have problem with padding)

ZEROS
OUTSIDE



OUTPUT

2	2	2	0
2	5	2	1
2	2	2	0
0	1	0	0



Convolution-Padding the Boundary

- There are two main reasons to pad convolutions
 - First, doing so helps us separate the matters of convolution and changing image sizes, so we have one less thing to remember.
 - Second, when we have more elaborate structures (e.g., skip connections), we want the tensors before and after a few convolutions to be of compatible size so that we can add them or take differences.

Looking Further with Depth and Pooling

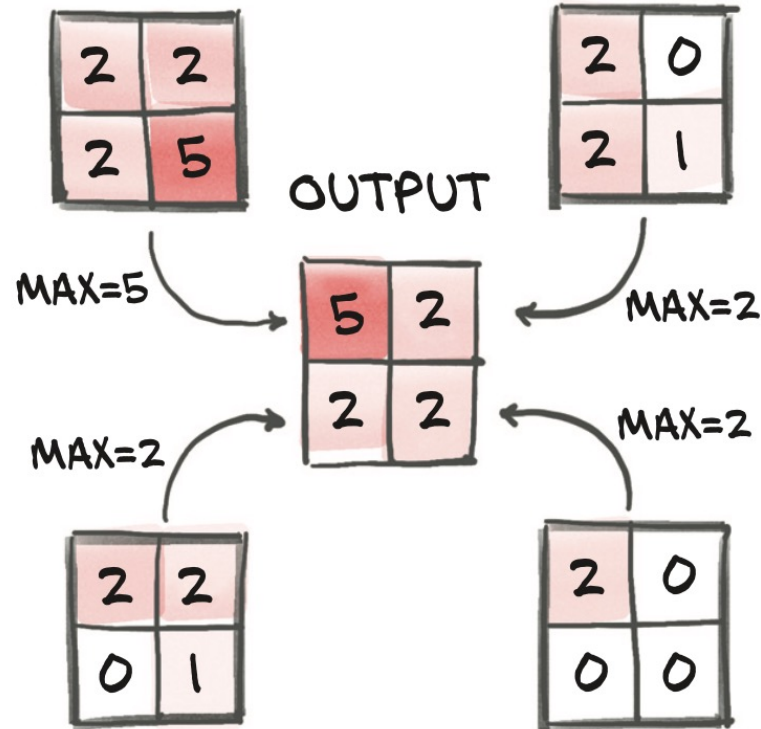
- The use of small kernels, like 3×3 , or 5×5 : that's peak locality, all right. What about the *big picture*?
- How are our networks going to be equipped to see those patterns with larger scope?
 - Downsampling: scaling an image by half is the equivalent of taking four neighboring pixels as input and producing one pixel as output.
 - *Average pooling: Average the pixels.*
 - *Max pooling: Take the maximum of the pixels*

Looking Further with Depth and Pooling

INPUT
(OUTPUT OF CONV + ACTIVATION)

2	2	2	0
2	5	2	1
2	2	2	0
0	1	0	0

MAXPOOL



- By keeping the highest value in the 2×2 neighborhood as the downsampled output, we ensure that the features that are found *survive* the downsampling.

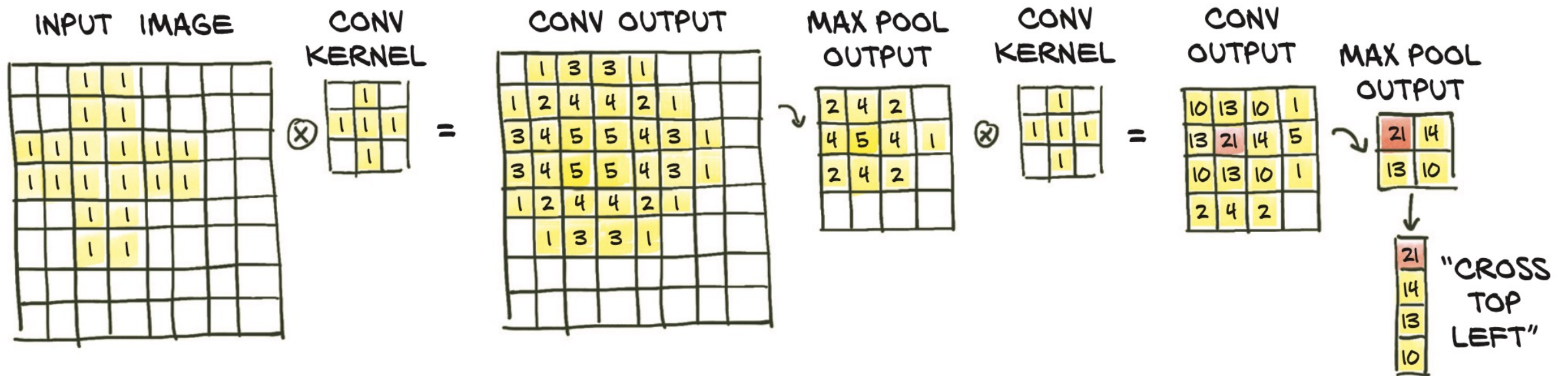
Looking Further with Depth and Pooling

- `nn.MaxPool2d` module: If we wish to downsample our image by half, we'll want to use a size of 2.

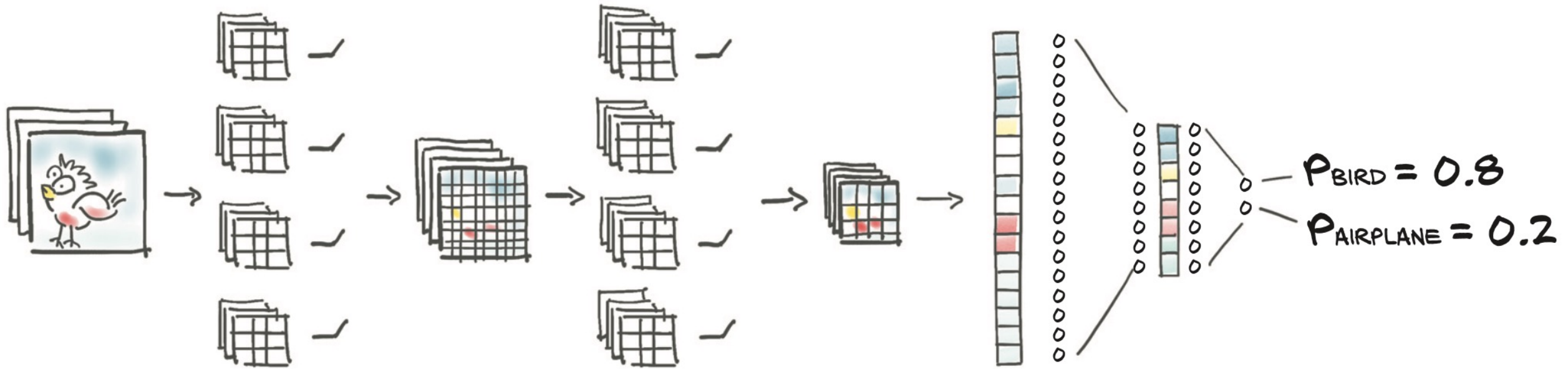
```
# In[21]:  
pool = nn.MaxPool2d(2)  
output = pool(img.unsqueeze(0))  
  
img.unsqueeze(0).shape, output.shape  
  
# Out[21]:  
(torch.Size([1, 3, 32, 32]), torch.Size([1, 3, 16, 16]))
```

Looking Further with Depth and Pooling

- Combining convolutions and downsampling
 - The first set of kernels operates on small neighborhoods on first-order, low-level features, while the second set of kernels effectively operates on wider neighborhoods, producing features that are compositions of the previous features.



Convolutional Neural Network

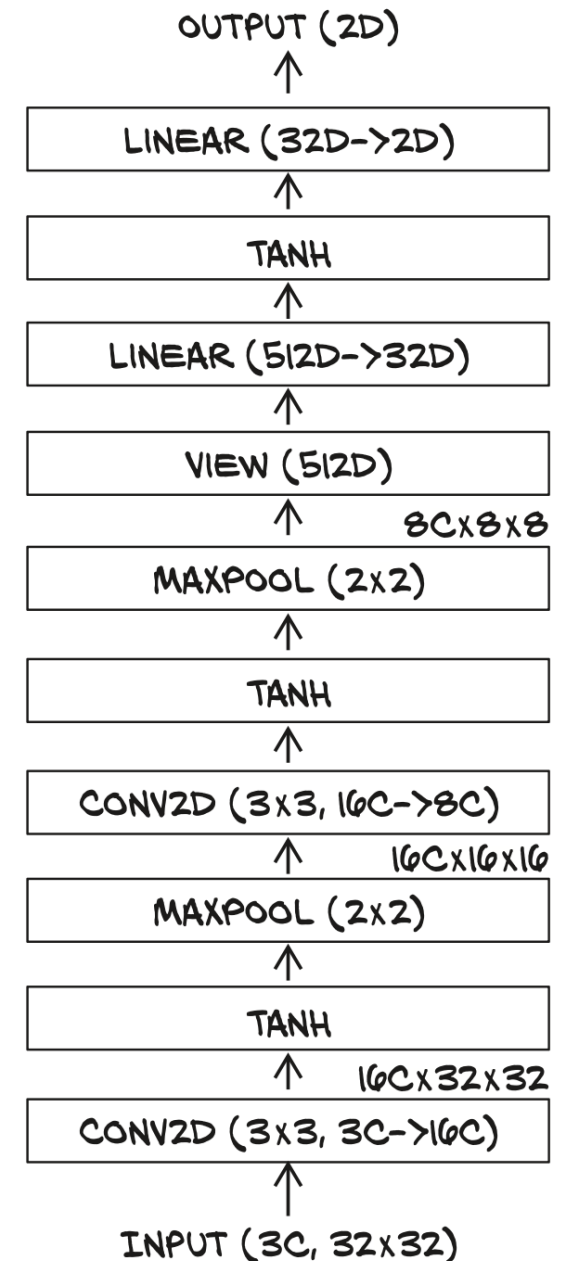


Convolutional Neural Network

```
# In[26]:
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.act1 = nn.Tanh()
        self.pool1 = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(16, 8, kernel_size=3, padding=1)
        self.act2 = nn.Tanh()
        self.pool2 = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(8 * 8 * 8, 32)
        self.act3 = nn.Tanh()
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = self.pool1(self.act1(self.conv1(x)))
        out = self.pool2(self.act2(self.conv2(out)))
        out = out.view(-1, 8 * 8 * 8)
        out = self.act3(self.fc1(out))
        out = self.fc2(out)
        return out
```

NET



Training on the GPU

```
for imgs, labels in train_loader:
```

```
    imgs = imgs.to(device=device)  
    labels = labels.to(device=device)
```

```
    outputs = model(imgs)  
    loss = loss_fn(outputs, labels)
```

```
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()
```

```
    loss_train += loss.item()
```

```
if epoch == 1 or epoch % 10 == 0:
```

```
    print('{} Epoch {}, Training loss {}'.format(  
        datetime.datetime.now(), epoch,  
        loss_train / len(train_loader)))
```

← **These two lines that move imgs and labels to the device we are training on are the only difference from our previous version.**

Training on the GPU

```
torch.manual_seed(0)
model = neural_network(32, 10).to(device=device)
optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()
```

← Moves our model (all parameters) to the GPU. If you forget to move either the model or the inputs to the GPU, you will get errors about tensors not being on the same device, because the PyTorch operators do not support mixing GPU and CPU inputs.

Improve the Performances

- ***Adding memory capacity: Width***

- The first dimension is the *width* of the network: the number of neurons per layer, or channels per convolution.

- ***Helping our model to converge and generalize: DROPOUT***

- The idea behind dropout is indeed simple: zero out a random fraction of outputs from neurons across the network, where the randomization happens at each training iteration.

- ***Going deeper to learn more complex structures: Depth***

- In regard to computer vision, a shallower network could identify a person's shape in a photo, whereas a deeper network could identify the person, the face on their top half, and the mouth within the face. Depth allows a model to deal with hierarchical information when we need to understand the context in order to say something about some input.